

南開大學
Nankai University

计算机学院

《人工智能导论》课程实验报告

机器人自动走迷宫

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

实验时间：2025 年 12 月 22 日 星期一

目录

1 问题重述 **2**
1.1 实验背景 . . . 2
1.2 实验要求 . . . 2
1.3 实验环境 . . . 2
1.4 注意事项 . . . 3
1.5 参考资料 . . . 3

2 设计思想 **3**
2.1 基础搜索算法（A*） . . . 3
2.1.1 状态与动作 . . . 3
2.1.2 评价函数与启发式函数 . . . 3
2.1.3 搜索流程 . . . 4
2.2 Deep Q-Learning 算法 . . . 4
2.2.1 状态与动作 . . . 4
2.2.2 经验回放 . . . 4
2.2.3 目标优化 . . . 4
2.2.4 奖励函数与动作选择策略 . . . 5

3 程序代码 **5**
3.1 基础搜索算法（A*） . . . 5
3.2 Deep Q-Learning 算法 . . . 6

4 测试结果 **9**

5 总结 **11**

1 问题重述

1.1 实验背景

在本实验中，要求分别使用基础搜索算法和 Deep Q-Learning 算法，完成机器人自动走迷宫。

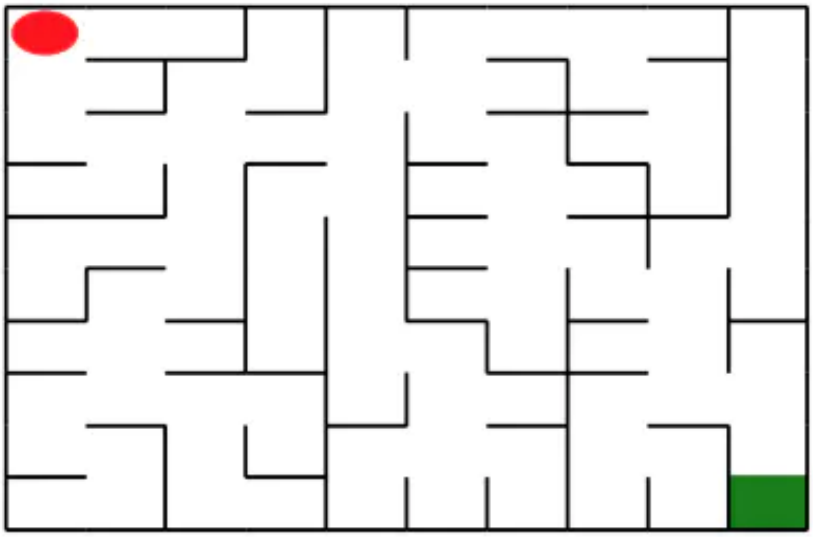


图 1.1.1: 机器人自动走迷宫

如上图 1.1.1 所示，左上角的红色椭圆既是起点也是机器人的初始位置，右下角的绿色方块是出口。

游戏规则为：从起点开始，通过错综复杂的迷宫，到达目标点（出口）。

在任一位置可执行动作包括：向上走 `'u'`、向右走 `'r'`、向下走 `'d'`、向左走 `'l'`。

执行不同的动作后，根据不同的情况会获得不同的奖励，具体而言，有以下几种情况：

- 撞墙
- 走到出口
- 其余情况

你需要实现基于基础搜索算法和 Deep Q-Learning 算法的机器人，使机器人自动走到迷宫的出口。

1.2 实验要求

- 使用 Python 语言。
- 使用基础搜索算法完成机器人走迷宫。
- 使用 Deep Q-Learning 算法完成机器人走迷宫。
- 算法部分需要自己实现，不能使用现成的包、工具或者接口。

1.3 实验环境

可以使用 Python 实现基础算法的实现，使用 Keras、PyTorch 等框架实现 Deep Q-Learning 算法。

1.4 注意事项

- Python 与 Python Package 的使用方式，可在右侧 API 文档中查阅。
- 当右上角的『Python 3』长时间指示为运行中的时候，造成代码无法执行时，可以重新启动 Kernel 解决（左上角『Kernel』-『Restart Kernel』）。

1.5 参考资料

1. 强化学习入门 MDP：https://zhuanlan.zhihu.com/p/25498081
2. Q-Learning 简单例子（英文）：http://mnemstudio.org/path-finding-q-learning-tutorial.htm
3. Q-Learning 简单解释（知乎）：https://www.zhihu.com/question/26408259
4. Deep Q-Learning 论文：https://files.momodel.cn/Playing%20Atari%20with%20Deep%20Reinforcement%20Learning.pdf

2 设计思想

2.1 基础搜索算法（A*）

本实验的基础搜索部分采用 A* 算法在迷宫网格图中搜索从起点到出口的可行路径。相比 DFS，A* 引入启发式函数，使搜索更倾向于朝目标方向扩展，从而减少无效节点扩展并提升搜索效率。

2.1.1 状态与动作

- **状态**：机器人在迷宫中的位置坐标 $s = (x, y)$。
- **动作**：四个离散动作集合 $A = \{u, r, d, l\}$，分别表示上、右、下、左移动。
- **合法动作**：通过 `maze.can_move_actions(cur)` 获取当前位置可执行动作，保证扩展时不会撞墙。

2.1.2 评价函数与启发式函数

A* 使用如下评价函数选择扩展顺序：

$$f(n) = g(n) + h(n), \tag{1}$$

其中 $g(n)$ 为从起点到节点 n 的真实代价，本实现使用步数累加（每走一步代价 $+1$）；$h(n)$ 为从节点 n 到目标节点的估计代价，本实现采用曼哈顿距离：

$$h(n) = |x - x_g| + |y - y_g|. \tag{2}$$

对应代码中 `_manhattan(a,b)`。

2.1.3 搜索流程

在 `my_search(maze)` 中，使用最小堆 `open_heap` 维护待扩展节点集合，堆元素存储 (f, g, pos)，每次取出 f 值最小的节点进行扩展。使用字典 `came_from` 记录父节点与动作映射：

$$\texttt{came_from[child]} = (\texttt{parent}, \texttt{action}), \tag{3}$$

当搜索到达出口位置时，从目标节点反向回溯动作序列并反转得到最终路径输出（即机器人执行的动作列表）。

综上，A* 在本任务中能够以较少节点扩展快速求得一条可行路径，并输出动作序列供机器人执行。

2.2 Deep Q-Learning 算法

强化学习部分采用 Deep Q-Learning 算法。该方法通过神经网络近似动作价值函数 $Q(s, a)$，并结合经验回放与目标网络提高训练稳定性。

2.2.1 状态与动作

- **状态**：当前位置坐标 $s = (x, y)$。为提升训练稳定性，将坐标按迷宫边长进行归一化：

$$\tilde{s} = \frac{s}{(\text{maze_size} - 1)}, \tag{4}$$

  对应代码中的 `_state_to_tensor()`。

- **动作**：$A = \{u, r, d, l\}$，网络输出对应四个动作的 Q 值。

神经网络 `_DQN` 结构为两层全连接网络（ReLU 激活），输入维度为 2，输出维度为 4，用于估计各动作的 $Q(s, a)$：

$$\texttt{Linear}(2 \to 64) \to \texttt{ReLU} \to \texttt{Linear}(64 \to 64) \to \texttt{ReLU} \to \texttt{Linear}(64 \to 4).$$

2.2.2 经验回放

每一步交互产生的经验元组 $(s, a, r, s', done)$ 存入 `ReplayDataSet`：

$$(s, a, r, s', done) \;\to\; \texttt{memory.add(...)}.$$

训练时通过 `random_sample(batch_size)` 随机采样小批量数据进行更新，以打破样本时间相关性并提高样本利用率。此外，考虑到经验不足时采样可能返回 `None`，代码中加入防御判断，避免解包异常导致程序中断。

2.2.3 目标优化

DQN 同时维护在线网络 Q_θ（`q_net`）与目标网络 Q_{θ^-}（`target_net`）。使用目标网络计算 TD 目标：

$$y = r + \gamma(1 - done) \max_{a'} Q_{\theta^-}(s', a'), \tag{5}$$

并用回归损失（SmoothL1/Huber Loss）最小化

$$\mathcal{L} = \|Q_\theta(s,a) - y\|.$$

代码中对应 `_train_step()` 的 `target` 计算与 `loss` 更新。目标网络每隔 `target_update_steps` 步同步一次参数：

$$\theta^- \leftarrow \theta.$$

2.2.4 奖励函数与动作选择策略

迷宫环境的奖励更像“代价”（例如到达终点为负值更优）。为适配 DQN 的“最大化回报”目标，训练时对奖励取负号，将代价转换为收益：

$$r_{\text{train}} = -r_{\text{env}}. \tag{6}$$

动作选择采用 ϵ-greedy 策略：

$$a = \begin{cases} \text{random action}, & \text{with prob. } \epsilon, \\ \arg\max_a Q_\theta(s,a), & \text{with prob. } 1-\epsilon. \end{cases} \tag{7}$$

同时对非法动作进行屏蔽（将非法动作的 Q 值置为极小），确保不会选择越界。

3 程序代码

3.1 基础搜索算法（A*）

```python
import heapq

_MOVE_MAP = {
    'u': (-1, 0),
    'r': (0, +1),
    'd': (+1, 0),
    'l': (0, -1),
}

def _manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def my_search(maze):
    start = tuple(maze.sense_robot())
    goal = tuple(maze.destination)
    if start == goal:
        return []

    open_heap = []
    heapq.heappush(open_heap, (_manhattan(start, goal), 0, start))

    came_from = {}
    g_score = {start: 0}
    closed = set()

```

```
    while open_heap:
        _, g, cur = heapq.heappop(open_heap)
        if cur in closed:
            continue
        closed.add(cur)

        if cur == goal:
            path = []
            node = cur
            while node in came_from:
                prev, act = came_from[node]
                path.append(act)
                node = prev
            path.reverse()
            return path

        for act in maze.can_move_actions(cur):
            dr, dc = _MOVE_MAP[act]
            nxt = (cur[0] + dr, cur[1] + dc)
            if nxt in closed:
                continue

            ng = g + 1
            if ng < g_score.get(nxt, 1e18):
                g_score[nxt] = ng
                came_from[nxt] = (cur, act)
                f = ng + _manhattan(nxt, goal)
                heapq.heappush(open_heap, (f, ng, nxt))

    return []
```

3.2 Deep Q-Learning 算法

```
import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from QRobot import QRobot
from ReplayDataSet import ReplayDataSet


class _DQN(nn.Module):
    def __init__(self, in_dim=2, hidden=64, out_dim=4):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden),
            nn.ReLU(),
            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, out_dim),
        )

    def forward(self, x):
        return self.net(x)
```

```
24
25
26 class Robot(QRobot):
27     def __init__(
28         self,
29         maze,
30         gamma=0.95,
31         lr=1e-3,
32         batch_size=64,
33         memory_size=50000,
34         epsilon_start=1.0,
35         epsilon_end=0.05,
36         epsilon_decay=0.995,
37         target_update_steps=200,
38         use_full_view=True,
39     ):
40         super(Robot, self).__init__(maze)
41         self.maze = maze
42         self.actions = ['u', 'r', 'd', 'l']
43
44         self.gamma = float(gamma)
45         self.batch_size = int(batch_size)
46
47         self.epsilon = float(epsilon_start)
48         self.epsilon_end = float(epsilon_end)
49         self.epsilon_decay = float(epsilon_decay)
50
51         self.target_update_steps = int(target_update_steps)
52         self.step_count = 0
53
54         self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
55
56         self.q_net = _DQN().to(self.device)
57         self.target_net = _DQN().to(self.device)
58         self.target_net.load_state_dict(self.q_net.state_dict())
59         self.target_net.eval()
60
61         self.optim = optim.Adam(self.q_net.parameters(), lr=lr)
62         self.loss_fn = nn.SmoothL1Loss()
63
64         self.memory = ReplayDataSet(max_size=memory_size)
65         if use_full_view:
66             try:
67                 self.memory.build_full_view(maze=self.maze)
68             except Exception:
69                 pass
70
71         self._maze_size = None
72         if hasattr(self.maze, "maze_size"):
73             self._maze_size = int(self.maze.maze_size)
74         elif hasattr(self.maze, "maze_data"):
75             self._maze_size = int(self.maze.maze_data.shape[0])
76
77     def _state_to_tensor(self, s):
78         s = np.array(s, dtype=np.float32).reshape(1, 2)
79         if self._maze_size is not None and self._maze_size > 1:
80             s = s / float(self._maze_size - 1)
81         return torch.from_numpy(s).to(self.device)
82
```

```
    def _select_action_dqn(self, state, valid_actions, epsilon):
        # epsilon-greedy
        if random.random() < epsilon and valid_actions:
            return random.choice(valid_actions)

        with torch.no_grad():
            q = self.q_net(self._state_to_tensor(state)).detach().cpu().numpy().reshape(-1)

        # mask 非法动作
        valid_idx = {self.actions.index(a) for a in valid_actions}
        for i in range(4):
            if i not in valid_idx:
                q[i] = -1e9

        return self.actions[int(np.argmax(q))]

    def _train_step(self):
        try:
            batch = self.memory.random_sample(self.batch_size)
        except Exception:
            return
        if batch is None:
            return

        states, actions, rewards, next_states, terminals = batch

        s = torch.tensor(states, dtype=torch.float32, device=self.device)
        ns = torch.tensor(next_states, dtype=torch.float32, device=self.device)
        a = torch.tensor(actions, dtype=torch.int64, device=self.device)

        r = -torch.tensor(rewards, dtype=torch.float32, device=self.device)
        done = torch.tensor(terminals, dtype=torch.float32, device=self.device)

        if self._maze_size is not None and self._maze_size > 1:
            s = s / float(self._maze_size - 1)
            ns = ns / float(self._maze_size - 1)

        q_sa = self.q_net(s).gather(1, a)

        with torch.no_grad():
            max_next_q = self.target_net(ns).max(dim=1, keepdim=True)[0]
            target = r + (1.0 - done) * self.gamma * max_next_q

        loss = self.loss_fn(q_sa, target)

        self.optim.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(self.q_net.parameters(), 10.0)
        self.optim.step()

        self.step_count += 1
        if self.step_count % self.target_update_steps == 0:
            self.target_net.load_state_dict(self.q_net.state_dict())

    def train_update(self):
        state = tuple(self.sense_state())
        valid_actions = self.current_state_valid_actions()

        action = self._select_action_dqn(state, valid_actions, self.epsilon)
```

```
        reward = float(self.maze.move_robot(action))
        next_state = tuple(self.sense_state())

        terminal = 1 if tuple(next_state) == tuple(self.maze.destination) else 0
        self.memory.add(state, self.actions.index(action), reward, next_state, terminal)

        self._train_step()

        self.epsilon = max(self.epsilon_end, self.epsilon * self.epsilon_decay)
        return action, reward

    def test_update(self):
        state = tuple(self.sense_state())
        valid_actions = self.current_state_valid_actions()

        action = self._select_action_dqn(state, valid_actions, epsilon=0.0)
        reward = float(self.maze.move_robot(action))
        return action, reward
```

4 测试结果

系统对我的代码面向四种样例进行了如下测试:

基础搜索算法

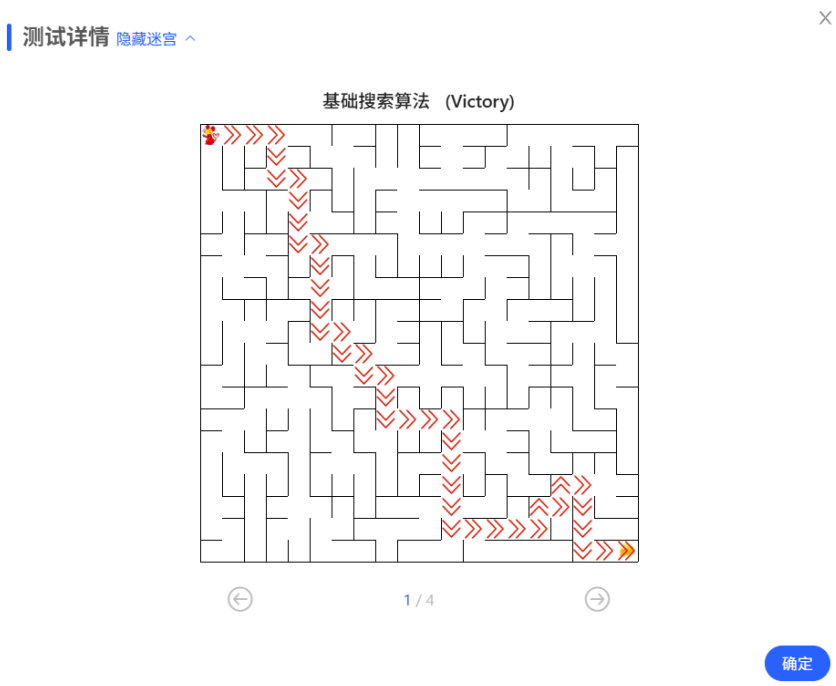


图 4.0.1: 基础搜索算法-测试结果

强化学习 level3



图 4.0.2: 强化学习 level3-测试结果

强化学习 level5

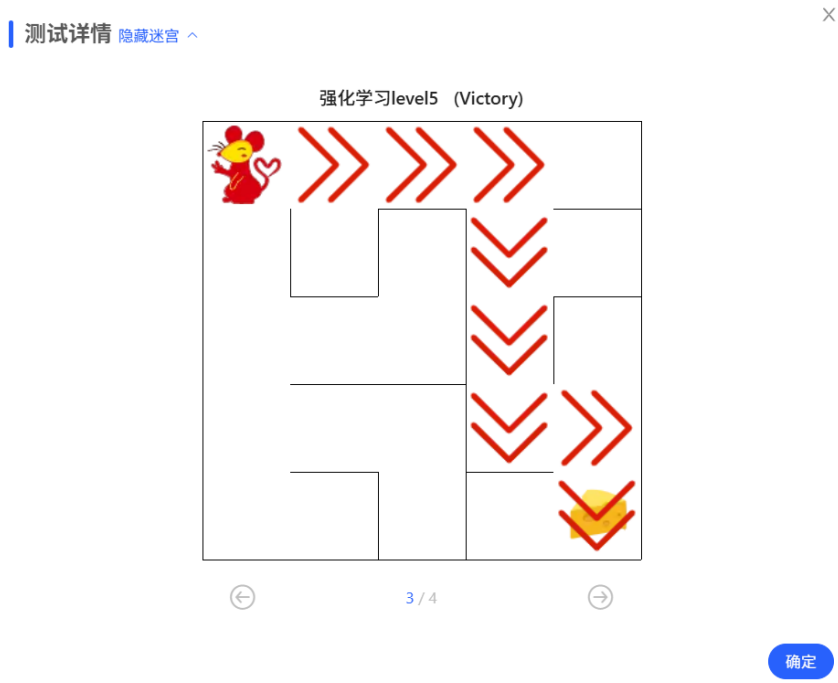


图 4.0.3: 强化学习 level5-测试结果

强化学习 level11



图 4.0.4: 强化学习 level11-测试结果

由在线测试结果可见，四种样例都顺利通过迷宫，且用时很短。

5 总结

本实验围绕“机器人自动走迷宫”任务，分别实现了基础搜索与强化学习两类方法。

基础部分采用 A* 搜索，将迷宫建模为网格图，通过曼哈顿距离启发函数引导搜索方向，能够较高效地找到从起点到终点的可行路径并输出动作序列。

强化学习部分采用 Deep Q-Learning，使用神经网络近似 $Q(s,a)$，结合经验回放与目标网络稳定训练，并采用 ϵ-greedy 策略在探索与利用之间进行权衡；同时针对环境奖励更偏“代价”的特点，对奖励进行了符号转换以适配最大化回报的优化目标。总体而言，A* 方法路径求解稳定且效率较高，Deep Q-Learning 方法具备学习与泛化能力，但在训练轮数有限时稳定性更依赖参数与样本质量，需要通过经验回放、目标网络等机制提升收敛表现。

本次实验中较为突出的挑战在于：强化学习在训练轮数有限且迷宫结构随机的情况下，策略收敛存在一定波动，不同随机种子可能导致结果差异；此外，迷宫奖励更接近“代价型”设定，若不进行合理的奖励处理，容易造成学习方向偏离目标。未来若面向更大规模迷宫或更复杂地图，算法将面临更高的计算与训练开销，需要在搜索剪枝、网络结构设计与训练样本效率等方面进一步改进，以获得更稳定且更优的路径规划效果。